"Study Group Finder"

1. Business Requirements Document (BRD)

Project Name: StudySync

Objective: An application for students to create or join study groups based on specific subjects or courses. Features include group scheduling, member limit management, and location/link posting.

Team Size: 6 members.

Functional Requirements:

- **User Authentication:** Users must be able to sign up and log in securely.
- **Group Creation:** Users must be able to create a study group by selecting a subject/course, group name, and description.
- **Group Discovery & Joining:** Users must be able to search for groups by subject and join one if space is available.
- **Scheduling & Location Posting:** Group creators must be able to set a session date/time and post a meeting location or video-call link.

Non-Functional Requirements:

- **Security:** Passwords must be hashed (Bcrypt).
- **Performance:** Group search results must load in under 1 second.
- **Responsiveness:** Must be fully functional on both mobile and desktop browsers.

2. Technical Design Document (TDD)

A. Tech Stack

Frontend: React (Vite), Tailwind CSS.

Backend: Node.js with Express.

Database: PostgreSQL with Prisma ORM.

Auth: JWT (JSON Web Tokens).

B. API Design

Method	Endpoint	Description
POST	/api/auth/register	Create a new user account.
POST	/api/auth/login	Authenticate and return JWT.

GET	/api/groups	Fetch all study groups (filter by subject).
POST	/api/groups	Create a new study group.
POST	/api/groups/:id/join	Join a group (checks member limit).
DELETE	/api/groups/:id	Remove a group (creator only).

C. Database Schema (Prisma)

```
model User {
  id        Int    @id @default(autoincrement())
  email    String  @unique
  password String
  createdGroups Group[] @relation("GroupCreator")
  joinedGroups  Group[] @relation("GroupMembers")
}
model Group {
  id         Int     @id @default(autoincrement())
  subject    String
  name       String
  memberLimit Int
  location   String?
  meetingLink String?
  scheduledAt DateTime
  creatorId   Int
  creator     User   @relation("GroupCreator", fields: [creatorId], referen
ces: [id])
  members     User[] @relation("GroupMembers")
}
```

D. Implementation Strategy

Phase 1 (Database): Setup Postgres and define the Prisma schema.

Phase 2 (Backend): Implement protected API routes. Use JWT middleware so only logged-in users can create or join groups.

Phase 3 (Frontend): Build the "Create Group" form and the "Group Search" page using React state to manage data from the API.

Phase 4 (Deployment): Deploy frontend on Vercel and backend on Render/Railway.

No code should be written until these files are committed and reviewed by the group.

3. Team Stories (6 Members — 1 Story Each)

Each story below is a complete, end-to-end feature (backend + frontend together). One team member owns one story from start to finish. When all 6 stories are done and merged, the full StudySync app is complete.

Story 1: Project Setup & Database

Owner: Member 1

As the Team, we want the project and database ready so that everyone else can start building their feature.

What to build:

- Create the Node.js + Express server and the React (Vite + Tailwind) app.
- Set up PostgreSQL and define the Prisma schema (User and Group models).
- Set up a shared GitHub repository with a basic folder structure (backend/, frontend/).
- Push the initial project so all 5 other members can pull it and start their story.

Acceptance Criteria: Both apps run locally with no errors; the Prisma migration runs successfully; the repo is shared with the team.

Story 2: User Authentication (Sign Up & Login)

Owner: Member 2

As a User, I want to sign up and log in securely so that I have my own private account.

What to build:

- Build POST /api/auth/register and POST /api/auth/login endpoints.
- Hash passwords with Bcrypt and return a JWT on successful login.
- Build the Sign Up and Login pages in React, connected to these endpoints.
- Store the JWT so the user stays logged in across pages.

Acceptance Criteria: A new user can sign up, log out, log back in, and reach a logged-in home page; wrong passwords are rejected.

Story 3: Group Creation

Owner: Member 3

As a User, I want to create a study group so that classmates can find and join it.

What to build:

- Build the POST /api/groups endpoint (subject, name, description, member limit).
- Build the "Create Group" form in React.

- Only allow logged-in users to create a group (use the JWT from Story 2).
- Show a success message and redirect after the group is created.

Acceptance Criteria: A logged-in user can fill the form and see their new group saved in the database.

Story 4: Group Discovery & Search

Owner: Member 4

As a User, I want to search for groups by subject so that I can find one that matches my course.

What to build:

- Build the GET /api/groups endpoint with an optional subject filter.
- Build the "Browse Groups" page listing all groups as cards.
- Add a search/filter input connected to the subject filter.
- Show group name, subject, and current member count on each card.

Acceptance Criteria: Typing a subject filters the list correctly; all existing groups are visible by default.

Story 5: Join Group & My Groups

Owner: Member 5

As a User, I want to join a group and see my groups so that I know where I belong.

What to build:

- Build the POST /api/groups/:id/join endpoint; reject the join if memberLimit is reached.
- Add a "Join" button on each group card from Story 4.
- Build a "My Groups" page showing groups the user created or joined.
- Disable/hide the Join button once a group is full.

Acceptance Criteria: Joining a full group is blocked with a clear message; joined groups appear under My Groups.

Story 6: Scheduling, Location & Group Management

Owner: Member 6

As a Group Creator, I want to set a schedule and location, and manage my group, so that members know when/where to meet and I can remove it if plans change.

What to build:

- Add scheduledAt, location, and meetingLink fields to the group detail view.
- Build a "Group Details" page showing the date/time and location/link.

- Build the DELETE /api/groups/:id endpoint (creator only).
- Add a "Delete Group" button visible only to the creator.

Acceptance Criteria: The schedule and location/link display correctly on the details page; only the creator can delete the group.